

C14 – Lab Assignment Aggregation and Composition

1. Objective

This lab explores two fundamental forms of object ownership in object-oriented programming: **aggregation** and **composition**. Both describe a "has-a" relationship between classes, but they differ in how tightly the lifetime of the owned object is bound to its owner.

By the end of this lab you will be able to:

- ▶ Implement aggregation in C++ using a pointer to an externally created object.
- ▶ Implement composition in C++ using an embedded object managed by the owner.
- ▶ Explain the lifetime and ownership rules that distinguish the two relationships.
- ▶ Read and interpret UML class diagrams showing aggregation and composition.

2. Background

2.1 Aggregation — "uses a"

In aggregation, one object holds a **reference or pointer** to another object that was created externally and can exist independently. The owner does not control the owned object's lifetime: destroying the owner does not destroy the referenced object.

Real-world analogy: a *Car* uses an *Engine*, but the engine can be removed, stored, and installed in a different car. The engine exists before and after any particular car.

C++ signature: the owner holds a `Engine*` (pointer) or `Engine&` (reference). The owner's destructor does **not** call `delete` on the pointer.

2.2 Composition — "owns a"

In composition, one object **owns** another object completely. The owned object is created by and lives inside the owner. When the owner is destroyed, the owned object is destroyed with it.

Real-world analogy: a *Library* owns its *Book* objects. If the library is destroyed by fire, its books are gone. The books have no independent existence outside that library. (Example2. A human and his brain.)

C++ signature: the owner holds an `Engine` by value (embedded object) or a pointer it allocates itself and deletes in its destructor.

2.3 Side-by-side comparison

	Aggregation	Composition
UML diamond	◇ Open	◆ Filled
Ownership	Weak (borrows)	Strong (owns)
Object lifetime	Independent	Tied to owner
C++ field type	Pointer / reference	Embedded value
Owner destructor	Does NOT delete	Must delete (if heap)

3. Class Diagrams

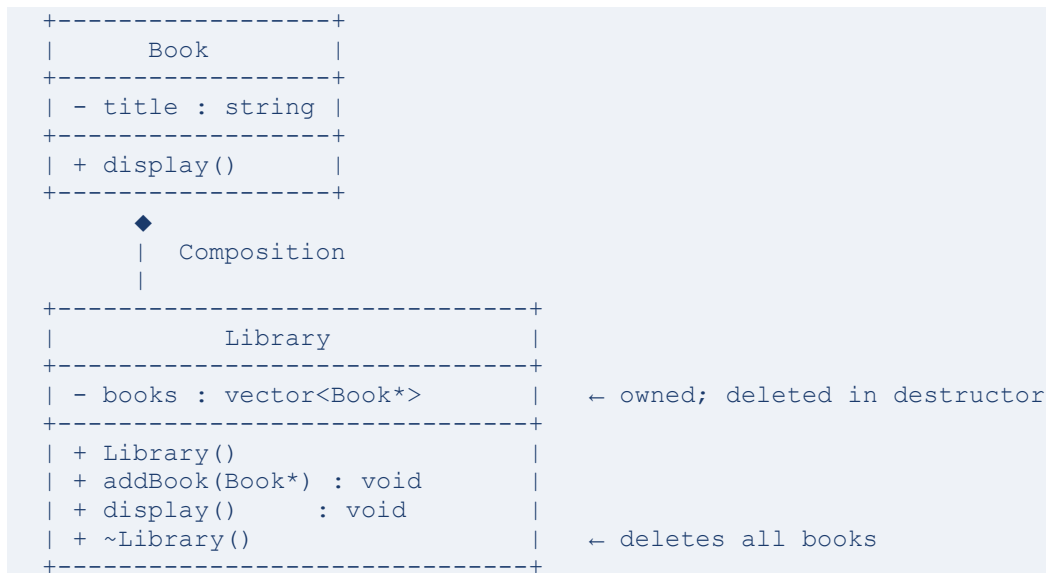
3.1 Diagram 1 — Aggregation: Car uses Engine

The open diamond (◇) on the Car side indicates that Car references an Engine it does not own. The Engine pointer is passed in from outside; the Car destructor must **not** delete it.

```
+-----+
|  Engine  |
+-----+
| + start() |
+-----+
      ◇
      | Aggregation
      |
+-----+
|      Car      |
+-----+
| - engine : Engine* | ← pointer (not owned)
+-----+
| + Car(Engine*)   |
| + startCar() : void |
+-----+
```

3.2 Diagram 2 — Composition: Library owns Books

The filled diamond (◆) on the Library side indicates full ownership. The Library allocates and destroys its Book objects. No Book can exist without its Library.



4. Part 1 — Aggregation: Car and Engine

4.1 Requirements

Implement two classes, `Engine` and `Car`, that model an aggregation relationship.

Engine class:

- One method: `start()` — prints a message confirming the engine has started.

Car class:

- **Field:** `engine : Engine*` (pointer, not owned)
- **Constructor:** `Car(Engine* e)` — stores the pointer; does NOT copy or delete the `Engine`.
- **Method:** `startCar()` — delegates to `engine->start()`.
- **Destructor:** `~Car()` — must NOT delete the engine pointer.

4.2 Starter code

```
Engine* engine = new Engine();    // Engine created OUTSIDE Car
Car* myCar = new Car(engine);    // Car receives a pointer — does not own it

myCar->startCar();                // delegates to engine->start()

delete myCar;                    // Car destroyed — engine is NOT deleted
delete engine;                   // Engine deleted separately by its true owner
```

Caution - Common mistake

Do not call `delete engine` inside `Car`'s destructor. The `Car` does not own the `Engine` — deleting it here would produce a double-free error when the external owner also calls `delete` on the same pointer.

5. Part 2 — Composition: Library and Book

5.1 Requirements

Implement two classes, `Book` and `Library`, that model a composition relationship.

Book class:

- **Field:** `title : string`
- **Constructor:** `Book(string title)`
- **Method:** `display()` — prints the title.

Library class:

- **Field:** `books : vector<Book*>` (owned; `Library` allocates and deletes all entries)
- **Method:** `addBook(Book*)` — adds a `Book` pointer to the collection.
- **Method:** `display()` — prints all book titles.
- **Destructor:** `~Library()` — must delete every `Book` pointer in the vector.

5.2 Starter code

```
Library myLibrary;

myLibrary.addBook(new Book("The C++ Programming Language"));
myLibrary.addBook(new Book("Clean Code"));
myLibrary.addBook(new Book("Design Patterns"));

myLibrary.display();

// myLibrary goes out of scope here.
// ~Library() is called automatically and deletes all three Book objects.
```

Key test for composition

Add a print statement to `~Book()`. When the `Library` is destroyed, you should see one destructor message per `Book` — confirming that the `Library` controls the lifetime of its `Books`.

6. Reflection Questions

Answer each question in 2–3 sentences:

- ▶ 1. What is the key structural difference in C++ between an aggregation field and a composition field?
- ▶ 2. Why must the Car destructor NOT delete the Engine pointer?
- ▶ 3. What would happen if the Library destructor did not delete its Book pointers?
- ▶ 4. Can an aggregated object be shared between two owners simultaneously? Can a composed object?

Explain why or why not.

- ▶ 5. In the UML diagrams, what does the direction of the diamond tell you about ownership?